



Prática de Desenvolvimento com Containers Avançado

R

esumo

No Módulo 11, vamos explorar o uso de containers de forma avançada, focando em práticas como **networking de containers**, **configuração de múltiplos containers com Docker Compose**, e **melhores práticas para produção**. Você aprenderá a criar ambientes que incluem múltiplos serviços (por exemplo, um backend em **Flask**, um banco de dados em **MySQL**, e um servidor de aplicação) e a orquestrá-los para garantir um ambiente eficiente e escalável.

1. Conceitos Avançados de Containers

1.1 Networking de Containers

O Docker cria automaticamente uma rede bridge para conectar containers. É possível criar redes customizadas para isolar ou conectar grupos de containers. A comunicação entre containers pode ser estabelecida usando seus nomes como aliases dentro da rede definida pelo Docker, garantindo uma comunicação clara e organizada. Isso é crucial quando trabalhamos com múltiplos serviços.

Exemplo de comando para criar uma rede:

```
docker network create minha-rede
```

1.2 Volumes e Persistência de Dados

Um dos principais desafios ao usar containers é garantir a persistência dos dados, já que os containers são efêmeros por natureza. Com os volumes, podemos montar diretórios do sistema host dentro do container, garantindo que os dados sejam mantidos mesmo após o container ser finalizado ou reiniciado.

Exemplo de comando para criar e montar um volume:

```
docker volume create meu-volume
```

```
docker run -d -v meu-volume:/dados-container minha-imagem
```

2. Orquestração com Docker Compose

O Docker Compose permite a definição e execução de múltiplos containers a partir de um único arquivo `docker-compose.yml`. Isso simplifica a configuração e gerenciamento de ambientes complexos que requerem vários serviços, como um ambiente típico de desenvolvimento com um servidor web, banco de dados, e um serviço de cache.

Estrutura Básica do Docker Compose

```
version: '3'
```

```
services:
```

```
web:
```

```
image: flask-app
```

```
build: .
```

ports:

- "5000:5000"

depends_on:

- db

db:

image: mysql:5.7

environment:

MYSQL_ROOT_PASSWORD: rootpassword

MYSQL_DATABASE: meu_banco

No exemplo acima, temos dois serviços: web e db. O serviço web depende do db, garantindo que o banco de dados seja iniciado antes da aplicação Flask.

3. Integração de Ferramentas Externas

3.1 Utilização de Nginx como Proxy Reverso

O que é Nginx? Nginx é um servidor web de código aberto que funciona como um proxy reverso, load balancer e cache HTTP. Originalmente, foi projetado para lidar com altas cargas de tráfego em sites de grande porte e hoje é amplamente utilizado por empresas de todos os tamanhos devido à sua eficiência e desempenho. Além disso, o Nginx é conhecido por sua capacidade de lidar com múltiplas conexões simultâneas de forma eficaz, tornando-o uma escolha popular para servir como proxy reverso.

Por que usar Nginx como Proxy Reverso? A utilização do Nginx como proxy reverso oferece vários benefícios, principalmente quando se trata de aplicações que precisam de escalabilidade, segurança e desempenho. Eis algumas razões principais:

1. **Distribuição de Tráfego e Load Balancing:** Nginx pode distribuir o tráfego de entrada entre vários servidores de backend, o que melhora o desempenho e a disponibilidade da aplicação. Isso é especialmente útil em ambientes com múltiplos containers ou microservices, já que o Nginx pode direcionar o tráfego de maneira eficiente para evitar sobrecarga em um único ponto.
2. **Melhoria de Segurança:** Ao usar Nginx como proxy reverso, ele atua como um intermediário entre o cliente e o servidor backend. Isso ajuda a proteger o servidor backend de acessos diretos, minimizando a exposição da aplicação a possíveis ataques. Nginx também oferece funcionalidades de segurança, como filtragem de IPs, controle de acesso e proteção contra ataques DDoS.
3. **Cache e Performance:** Nginx pode armazenar em cache respostas do backend, o que reduz a carga do servidor e melhora o tempo de resposta para os usuários finais. Essa funcionalidade é crucial para aplicações que lidam com dados estáticos, como imagens, arquivos JavaScript e CSS.
4. **Terminação de SSL/TLS:** Nginx pode ser configurado para realizar a terminação SSL/TLS, o que significa que ele lida com a criptografia e descriptografia do tráfego HTTPS. Isso libera o backend dessa tarefa, resultando em melhor desempenho e simplificação da configuração de segurança.
5. **Reescrita de URLs e Controle de Rotas:** Com Nginx, você pode reescrever URLs, configurar redirecionamentos e gerenciar rotas de forma flexível. Isso é útil em cenários onde há múltiplos serviços rodando em diferentes portas ou quando você precisa de regras específicas de roteamento.

Imagine que você tenha uma aplicação Flask rodando em um container no Docker na porta 5000 e um servidor Nginx configurado para escutar a porta 80 (HTTP). O Nginx pode ser configurado para receber todas as

solicitações HTTP na porta 80 e redirecioná-las para a aplicação Flask na porta 5000. Isso cria uma camada intermediária que gerencia o tráfego, permitindo que o Nginx realize a filtragem, cache e gerenciamento de segurança.

Aqui está um exemplo de como seria a configuração de um arquivo `nginx.conf` para atuar como proxy reverso para uma aplicação Flask:

```
server {  
listen 80;  
  
server_name exemplo.com;  
  
location / {  
proxy_pass http://localhost:5000;  
proxy_set_header Host $host;  
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
proxy_set_header X-Forwarded-Proto $scheme;  
}  
}
```

Neste exemplo:

- O Nginx escuta na porta 80 e redireciona todo o tráfego para a aplicação Flask rodando em `localhost:5000`.
- O `proxy_set_header` garante que os cabeçalhos originais da solicitação sejam preservados e passados para o backend.

Kubernetes: O Que é e sua importância

O que é Kubernetes? Kubernetes, frequentemente abreviado como “K8s,” é uma plataforma de código aberto que automatiza a implantação, o dimensionamento e a gestão de aplicações em containers. Desenvolvido inicialmente pelo Google e agora mantido pela Cloud Native Computing Foundation (CNCF), Kubernetes é uma das tecnologias mais amplamente adotadas para orquestração de containers em ambientes de produção. Ele oferece uma maneira eficiente de gerenciar aplicativos em containers em diferentes ambientes, garantindo que eles funcionem de forma confiável, segura e escalável.

Por que Kubernetes é importante? Kubernetes é importante por várias razões:

1. Escalabilidade Automática: Uma das maiores vantagens do Kubernetes é a capacidade de escalar automaticamente suas aplicações com base na demanda. Isso significa que, quando há um aumento no tráfego ou na carga de trabalho, o Kubernetes pode adicionar mais containers para lidar com a demanda. Da mesma forma, ele pode reduzir a escala quando a carga diminui, otimizando o uso de recursos e custos.

2. Alta Disponibilidade e Resiliência: Kubernetes é projetado para garantir que suas aplicações estejam sempre disponíveis. Ele pode detectar quando um container falha e automaticamente reiniciá-lo ou substituí-lo, garantindo que os serviços permaneçam ativos e funcionando mesmo em caso de falhas.

3. Orquestração e Automação: Kubernetes gerencia e orquestra containers em diferentes servidores e ambientes, automatizando tarefas como

implantação, atualização, monitoramento e balanceamento de carga. Isso reduz significativamente o esforço manual, permitindo que as equipes de desenvolvimento e operações se concentrem em outras tarefas mais estratégicas.

4. Flexibilidade e Portabilidade: Kubernetes é compatível com diferentes ambientes de nuvem e on-premises, tornando mais fácil mover aplicações entre diferentes provedores de nuvem ou infraestruturas internas sem grandes alterações. Isso oferece às empresas a flexibilidade de escolher o ambiente que melhor se adapta às suas necessidades.

Como Prometheus e Grafana Interagem com Kubernetes?

A integração de Prometheus e Grafana com Kubernetes é um dos cenários de monitoramento mais comuns em ambientes de infraestrutura modernos, especialmente em organizações que gerenciam grandes clusters de containers.

1. Coleta de Métricas em Kubernetes com Prometheus:

- **Prometheus Operator:** O Prometheus Operator é uma ferramenta que facilita a implantação e o gerenciamento do Prometheus em clusters Kubernetes. Ele permite que você defina “ServiceMonitors” para especificar quais métricas devem ser coletadas de diferentes serviços e containers dentro do cluster.
- **Kube-State-Metrics:** Este componente coleta informações sobre o estado dos recursos do Kubernetes, como Pods, Deployments e Services, e expõe essas métricas em um formato que o Prometheus pode coletar.
- **Node Exporter:** Cada nó do cluster Kubernetes pode executar um Node Exporter que coleta métricas do sistema operacional, como uso de CPU,

memória e I/O. O Prometheus coleta esses dados para obter insights sobre a saúde do cluster como um todo.

2. Visualização com Grafana:

- O Grafana se conecta ao Prometheus como uma fonte de dados e permite a criação de dashboards interativos que mostram métricas em tempo real do cluster Kubernetes. Por exemplo, você pode visualizar o uso de CPU e memória de cada container, a quantidade de tráfego de rede ou a taxa de erros de uma aplicação.
- Além disso, o Grafana permite que você crie alertas personalizados que notificam a equipe quando os recursos do Kubernetes estão sobrecarregados ou quando há problemas de desempenho.

Importância de Entender Kubernetes, Prometheus e Grafana em Ambientes de Produção

Para quem busca se tornar um profissional disruptivo e nexialista, é essencial compreender como Kubernetes, Prometheus e Grafana se integram, pois essa combinação de tecnologias é amplamente utilizada em ambientes de produção em empresas que buscam inovação e eficiência. No entanto, o mais importante é entender que essas ferramentas não são uma solução universal. Cada empresa ou cliente tem requisitos específicos, e é crucial adaptar a abordagem de acordo com o cenário de negócio e a complexidade do ambiente.

Como Decidir Quando e Como Usar Essas Tecnologias?

1. Análise do Cenário do Cliente: Antes de implementar Kubernetes, Prometheus e Grafana, é fundamental entender o contexto do cliente ou

empresa. Perguntas como “Qual é a escala atual da infraestrutura?” ou “Qual é o nível de complexidade que a equipe pode gerenciar?” ajudam a determinar se Kubernetes é a solução ideal.

2. **Avaliação de Custo e Complexidade:** Kubernetes pode adicionar uma camada de complexidade e custo que pode não ser necessária para empresas menores ou para aplicações simples. É importante avaliar se o investimento em tempo e recursos para aprender e implementar Kubernetes realmente trará benefícios à operação.

3. **Necessidade de Escalabilidade e Flexibilidade:** Se a aplicação exige escalabilidade dinâmica, alta disponibilidade, e uma arquitetura baseada em microsserviços, Kubernetes se torna uma escolha adequada. Em casos mais simples, o uso de containers com Docker pode ser suficiente sem a necessidade de Kubernetes.

4. Melhores Práticas para Produção

1. Mantenha as imagens pequenas: Use imagens base minimalistas e exclua arquivos temporários após a instalação de pacotes.

2. Configure variáveis de ambiente com segurança: Nunca inclua credenciais diretamente no Dockerfile ou docker-compose.yml. Utilize arquivos .env ou sistemas de gerenciamento de segredos.

3. Limite o uso de privilégios: Sempre que possível, evite o uso da flag --privileged e execute o container com usuários não-root.

4. Automatize o build e deploy: Utilize pipelines CI/CD (por exemplo, Jenkins ou GitLab CI) para automatizar a criação, teste e implantação de imagens.

Dicas Práticas

- Utilize o comando `docker-compose up --build` para construir e iniciar todos os containers definidos no arquivo `docker-compose.yml`.
- Explore outras ferramentas de orquestração, como **Kubernetes**, para casos de uso mais complexos e escaláveis.
- Aproveite o potencial do ChatGPT para esclarecer dúvidas ou gerar scripts de Docker customizados.

Conteúdo Bônus

Título: Docker Compose na prática (Fácil)

Para quem está começando com **Docker Compose** e quer aprender de forma prática, o canal **Ayrton Teshima - Programador a Bordo** apresenta uma excelente aula introdutória. Neste vídeo, você aprenderá a transformar comandos individuais de `docker build` e `docker run` em um arquivo `docker-compose.yml`, permitindo rodar toda a estrutura do projeto com um único comando: `docker-compose up -d`.

Este conteúdo é ideal para quem quer otimizar e automatizar o gerenciamento de containers Docker, aproveitando ao máximo a simplicidade e eficiência que o Docker Compose proporciona para projetos com múltiplos serviços.

Referências

- **Docker Documentation.** Docker Networking. Disponível em: <https://docs.docker.com/network/>. Acesso em: 25 set. 2024.

- **NGINX.** How to Use NGINX as a Reverse Proxy with Docker. Disponível em: <https://www.nginx.com/resources/admin-guide/nginx-reverse-proxy/>. Acesso em: 25 set. 2024.

- **Prometheus Documentation.** Introduction to Prometheus. Disponível em: <https://prometheus.io/docs/introduction/overview/>. Acesso em: 25 set. 2024.

- **Grafana Documentation.** Getting Started with Grafana. Disponível em: <https://grafana.com/docs/grafana/latest/getting-started/>. Acesso em: 25 set. 2024.

Atividade Prática 11 - Prática de Desenvolvimento com Containers Avançado

Título da Prática: Orquestração de Containers com Kubernetes

Objetivos:

- Compreender o uso do Kubernetes para orquestrar aplicações containerizadas.
- Configurar e gerenciar múltiplos contêineres para garantir alta disponibilidade e escalabilidade da aplicação.
- Utilizar técnicas de orquestração para aprimorar o desempenho e a resiliência da aplicação em um ambiente de produção.

Materiais, Métodos e Ferramentas:

- Kubernetes instalado ou acesso a um ambiente de nuvem com suporte para Kubernetes (como Google Kubernetes Engine ou Minikube para

simulação local).

- Imagens de contêineres da aplicação previamente criadas (podem ser simples contêineres Docker).
- Documentação de comandos *kubectl* para manipulação e monitoramento dos clusters.

Atividade Prática

Roteiro

1º. Passo) **Configuração do Cluster Kubernetes:**

- Inicie um cluster Kubernetes utilizando Minikube (local) ou uma plataforma de nuvem.
- Verifique se o cluster está ativo e pronto para receber os containers da aplicação, utilizando o comando *kubectl cluster-info*.

2º. Passo) **Implantação dos Containers:**

- Utilize o *kubectl* para criar um arquivo de configuração YAML para o deployment da aplicação.
- Configure o deployment com múltiplas réplicas para garantir alta disponibilidade e resiliência.
- Execute o comando *kubectl apply -f [nome_do_arquivo].yaml* para implantar a aplicação no cluster.

3º. Passo) **Configuração do Balanceamento de Carga e Escalabilidade:**

- Crie um serviço LoadBalancer para distribuir o tráfego entre as réplicas da aplicação, utilizando o comando *kubectl expose*.

- Configure o autoscaler para ajustar automaticamente o número de réplicas de acordo com a carga, usando `kubectl autoscale deployment [nome_do_deployment] --min=2 --max=5 --cpu-percent=80`.

4º. Passo) **Monitoramento e Ajustes:**

- Monitore o status dos containers e os recursos consumidos utilizando `kubectl get pods`, `kubectl top pod`, e `kubectl describe`.
- Identifique possíveis melhorias na configuração, como o ajuste de recursos e limitação de CPU/memória para otimização de desempenho.

Gabarito Atividade Prática

Exemplo de Respostas:

1. **Configuração do Cluster:** O comando `kubectl cluster-info` confirmou que o cluster está ativo e funcional, com conectividade para gerenciar os pods da aplicação.
2. **Implantação e Réplicas:** A aplicação foi implantada com sucesso utilizando três réplicas para garantir alta disponibilidade. Todas as réplicas estão funcionando e distribuindo o tráfego de forma equilibrada.
3. **Balanceamento e Escalabilidade:** O serviço LoadBalancer foi configurado corretamente, permitindo que o tráfego seja distribuído entre as réplicas. O autoscaler ajusta as réplicas conforme o aumento da carga, otimizando o uso de recursos.
4. **Recomendações:**

- Aumentar a capacidade de memória e CPU para cada pod, garantindo que o autoscaler responda rapidamente a picos de uso.
- Configurar uma política de limite de recursos para evitar consumo excessivo em momentos de alta demanda.

Ir para exercício